

OPERATIONAL AUDIT — PHASE 1

Mobile Checklist Communication Verification

RentnKing iOS app ↔ Kabba backend (Laravel) · Prepared 2026-08-03

Phase 1 Audit: Mobile Checklist Communication Verification

Scope: RentnKing iOS app ↔ Kabba backend (Laravel). Verifying that every checklist completed on a mobile device actually reaches the server and executes its required downstream actions — not whether the mobile UI *says* it did.

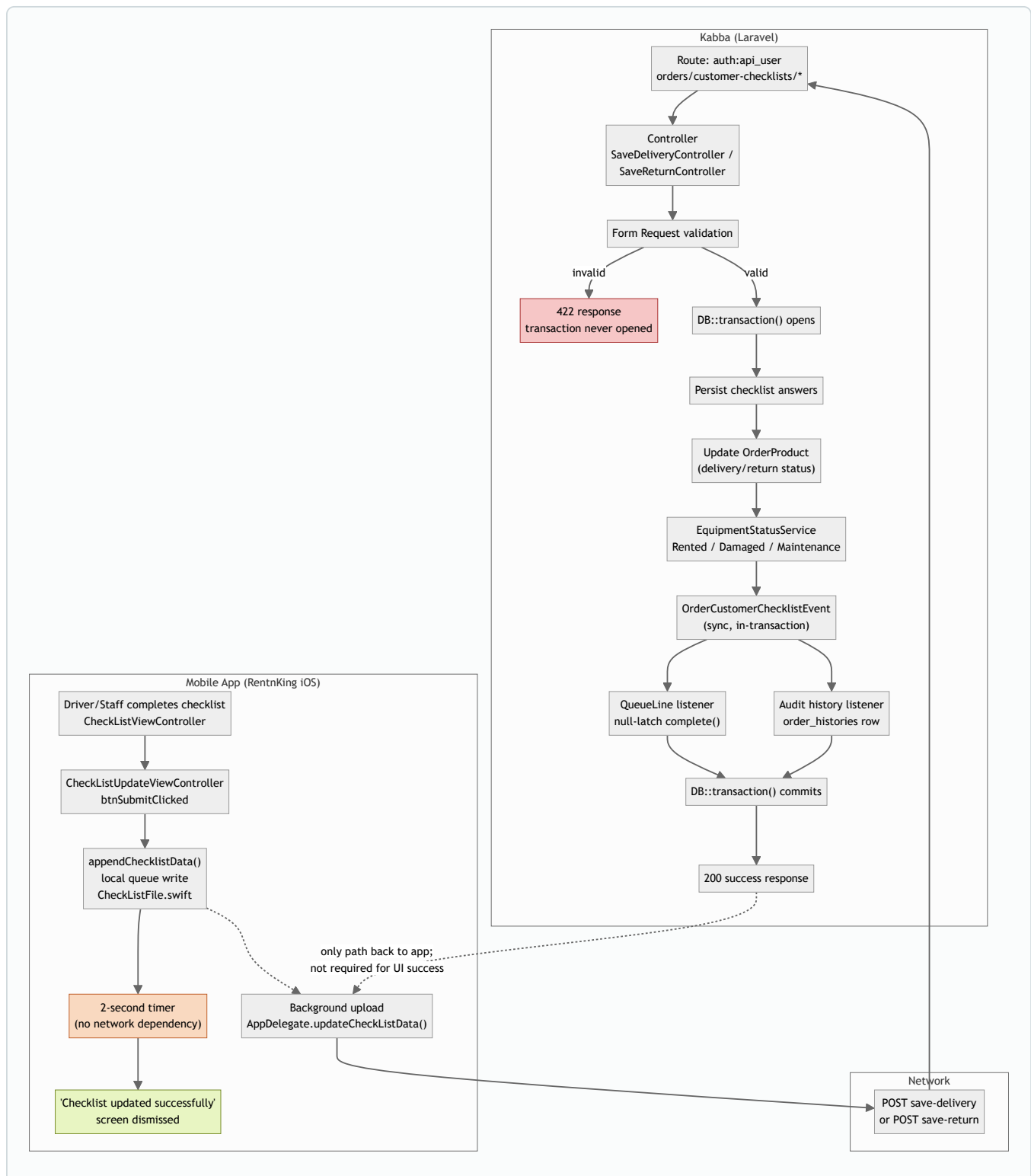
Repos audited: `mobileapp` (iOS/Swift, `RentnKing` target) and `kabba` (Laravel, `D:\Apps\laragon\www\2026\kabba`).

Bottom line

The mobile app cannot be trusted as a source of truth for "did this checklist go through." It reports success on a fixed 2-second timer, before the network request is even guaranteed to have started, and offers no dedup protection when a user or the retry queue resubmits. The backend, once a request actually arrives, is well-built — transactional, idempotent on Queue Line completion — with one confirmed workflow gap (Rental Ready is never auto-triggered after a return) and one silent-failure pattern (damage/fuel billing charges can fail without surfacing to anyone).

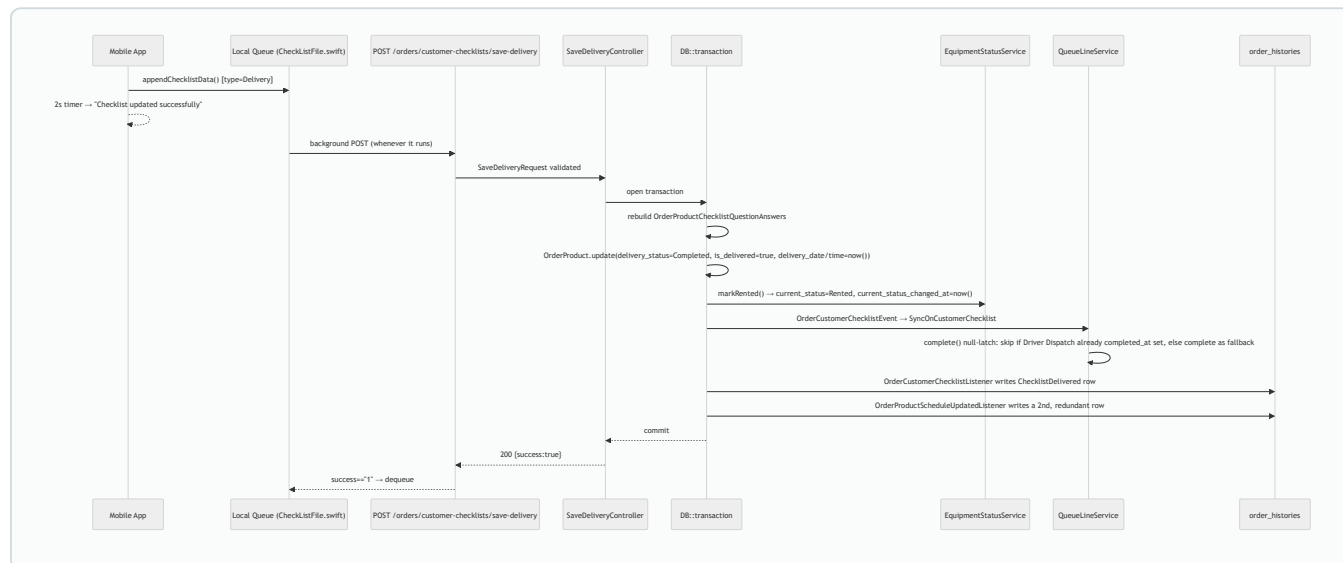
Both checklist families — delivery/pickup and return — funnel through **two backend endpoints total**, not six. There is no dedicated "in-store pickup" endpoint; pickup and truck delivery share `save-delivery`, distinguished only by whether a driver dispatch checklist ran first.

1. Communication diagram



2. Checklist processing maps

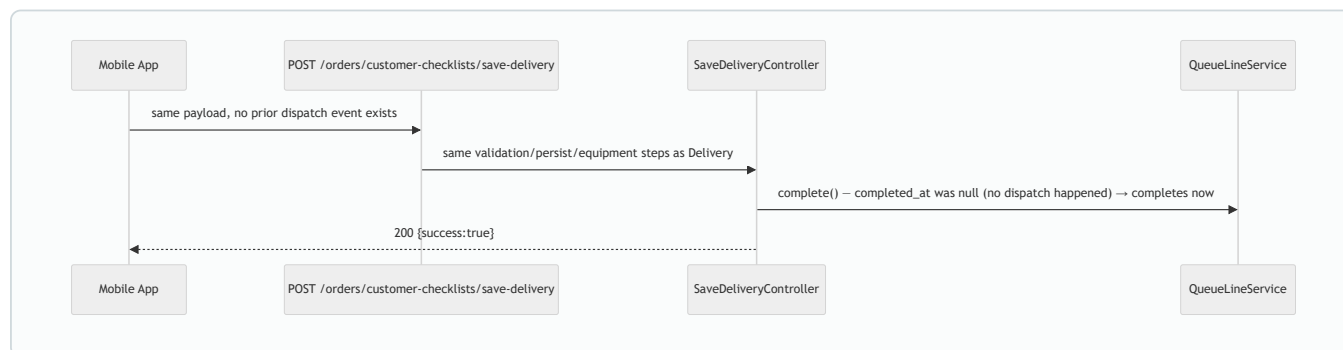
2a. Truck Delivery Customer Checklist



Rental start timing is not a discrete field/service — it is implicitly `delivery_date / delivery_time` plus `equipment.current_status_changed_at`, both stamped inside the same update. There is no formal rental-duration clock separate from these.

2b. In-Store Pickup Customer Checklist

Identical backend path to Truck Delivery (same controller, same route — `save-delivery`). The only functional difference: because no driver dispatch checklist ran first, the Queue Line null-latch in `QueueLineService::complete()` finds `completed_at` still null and **this checklist is what completes it**, rather than leaving it unchanged.



There is no route, controller, or code path literally named "pickup" — this is an inferred behavior resting entirely on the *absence* of a prior dispatch event, which is undocumented at the route layer.

2c. Customer Return Checklist



Syntax error in text
mermaid version 10.9.6

Confirmed gap: Rental Ready is a fully separate, manually-triggered endpoint/checklist. Nothing in the return path queues, flags, or notifies that a unit needs Rental Ready inspection — a technician must independently know to open it. `RentalReadyEligibility::canInspect()` only checks the unit isn't currently `Rented` ; it does not require or detect a prior return.

3. Source file inventory

Mobile app (RentnKing iOS)

Role	File
Checklist entry/submit UI	<code>RentnKing/Modules/TABBAR/Home Model/Order Model/CheckList/CheckListViewController.swift</code>
Submission logic, 2s timer, "success" UI	<code>RentnKing/Modules/TABBAR/Home Model/Order Model/CheckList/CheckListUpdateViewController.swift</code>
Endpoint URLs	<code>RentnKing/GlobalMain.swift (updateDeliveryCheckList , updateReturnCheckList)</code>
Dispatch to correct endpoint by type	<code>RentnKing/AppDelegate.swift (updateCheckList , updateCheckListData , appDataDidFail)</code>
Local offline queue (JSON/UserDefaults)	<code>RentnKing/Core/FileData Helper/CheckListFile.swift</code>
Network/response validation helper	<code>RentnKing/Core/.../WebServiceHelper.swift</code>
Driver-side dispatch queues (for contrast — these dedup, customer checklist queue doesn't)	<code>SyncDriverChecklist.swift , SyncDeliveryPickupInputs.swift</code>
Queue Line UI (delivery/pickup label only)	<code>.../Queue Line Model/QueueLineViewController.swift</code>

Backend (Kabba, Laravel)

Role	File
Route table	routes/api/admin/v1/orders/routes.php (lines 66-73), routes/api/admin/v1/routes.php (auth middleware)
Delivery/Pickup controller	app/Http/Controllers/Api/Admin/V1/Orders/CustomChecklists/SaveDeliveryController.php
Delivery/Pickup validation	app/Http/Requests/Api/Admin/V1/Orders/CustomChecklists/SaveDeliveryRequest.php
Return controller	app/Http/Controllers/Api/Admin/V1/Orders/CustomChecklists/SaveReturnController.php
Return validation	app/Http/Requests/Api/Admin/V1/Orders/CustomChecklists/SaveReturnRequest.php
Shared validation base (failure → 422)	app/Http/Requests/ApiBaseFormRequest.php
Order / order product models	app/Models/Orders/OrderProduct.php , app/Models/Orders/OrderProductChecklistQuestionAnswers.php , app/Models/Orders/OrderHistory.php
Equipment status transitions	app/Services/Equipment/EquipmentStatusService.php , app/Models/ChecklistManagement/EquipmentChecklist/EquipmentStatusLog.php , app/Enums/Equipments/EquipmentCurrentStatus.php
Queue Line completion (idempotent)	app/Services/QueueLine/QueueLineService.php , app/Listeners/QueueLine/SyncOnCustomerChecklist.php , app/Listeners/QueueLine/SyncOnScheduleUpdate.php
Events	app/Events/Admin/Orders/OrderCustomerChecklistEvent.php , app/Events/Admin/Orders/OrderProductScheduleUpdated.php
Audit history listeners	app/Listeners/Activities/Admin/Orders/OrderCustomerChecklistListener.php , app/Listeners/Activities/Admin/Orders/OrderProductScheduleUpdatedListener.php
Billing (return damage/fuel charges)	app/Services/BillingEngine.php , app/Services/ChargeService.php , app/Services/ChargeTaxCalculator.php , app/Http/DataObjects/BillingChargeRequest.php
Wait-list side effect on return	app/Services/WaitList/WaitListMatcher.php
Rental Ready (separate, not auto-triggered)	app/Http/Controllers/Api/Admin/V1/Orders/RentalReadyChecklists/SaveController.php , app/Services/ChecklistManagement/RentalReadyInspectionService.php , app/Services/ChecklistManagement/RentalReadyEligibility.php
Enums	app/Enums/Orders/OrderCustomerChecklistType.php , app/Enums/Orders/OrderHistoryAction.php
Relevant tests	tests/Feature/CustomChecklists/ChecklistTransactionTest.php , tests/Feature/QueueLine/QueueLineReleaseAndCompletionTest.php

No Laravel queued jobs (`ShouldQueue` , `->dispatch()`) exist anywhere in either checklist path — every event listener runs synchronously inside the same request/transaction.

4. Communication verification

Workflow	Checklist persisted	Order updated	Equipment updated	Queue Line updated	Rental start	Rental Ready initiated	Audit history
Truck Delivery	✓ transactional	✓ <code>delivery_status</code> , <code>is_delivered</code>	✓ → Rented	✓ null-latch fallback, correctly skips if dispatch already completed	⚠ implicit only (no dedicated clock field)	n/a	✓ (written twice, redundant)
In-Store Pickup	✓ same controller as Delivery	✓ same	✓ same	✓ this checklist is the sole trigger	⚠ implicit only	n/a	✓
Customer Return	✓ transactional	✓ <code>pickup_status</code> , <code>is_returned</code>	✓ → Damaged/Maintenance	n/a	n/a	✗ never triggered — confirmed gap	✓ (single write; no 2nd listener fires here)

All three are wrapped in `DB::transaction()`, so **once a request reaches the controller**, partial failure is not possible server-side — it all commits or all rolls back. The verification gap is entirely upstream of the controller: whether the request reaches it at all, and whether the app knows if it didn't.

5. Failure points

Ranked by how far the failure gets before something silently looks fine to a human.

1. Mobile UI reports success independent of the network call — the most severe gap.

`CheckListUpdateViewController.swift:391` starts a hard-coded `DispatchQueue.main.asyncAfter(2s)` immediately after the *local* queue write, then shows "Checklist updated successfully" and navigates away. The actual POST fires later in the background. If the device is offline, the server rejects it, or the app is killed before the background upload runs, **the user has already seen and dismissed a success screen for a checklist the backend never received.**

2. No duplicate-submission protection on the customer checklist queue.

`CheckListFile.swift`'s `appendChecklistData()` just appends — no idempotency key, no local dedup. Contrast with the driver-side queues (`SyncDriverChecklist.swift`, `SyncDeliveryPickupInputs.swift`), which explicitly remove any existing pending record for the same product before appending. A user re-opening and resubmitting a checklist before the first queued entry uploads produces two queued POSTs to the same endpoint.

3. Retry queue can silently give up.

Server-rejected items retry up to `kMaxSyncAttempts = 10` (`SyncDriverChecklist.swift:12`) then are dropped — with no user-facing indication that a "completed" checklist was ultimately never delivered.

4. Rental Ready is never automatically initiated after a return.

Confirmed by exhaustive grep across `app/Observers`, `app/Listeners`, `app/Events`, `app/Jobs` — no dispatch exists on the return path. Equipment

can sit in `Maintenance` / `Damaged` status indefinitely until a technician separately remembers to run the Rental Ready checklist.

5. **Damage/fuel billing charges on return fail silently by design.** `SaveReturnController.php` wraps `BillingEngine::charge()` calls in try/catch that only logs to the `billing_engine` channel — the return still returns 200 even if the associated charge silently fails to record.
6. **WaitListMatcher failures on return are also swallowed** (logged only), intentionally, so they never block a return — but this means wait-list matching can silently break with zero visibility.
7. **Redundant audit rows on delivery/pickup.** Two listeners (`OrderCustomerChecklistListener` and `OrderProductScheduleUpdatedListener`) both write an `order_histories` row per delivery/pickup submission — not a correctness bug, but noise in the audit trail that could mislead a future investigation into thinking two separate events occurred.
8. **In-store pickup has no explicit identity in the codebase.** It is entirely inferred from the absence of a prior dispatch event. Anyone auditing, testing, or extending this later has nothing at the route/controller layer telling them "this is the pickup case" — raising the risk that a future change to the dispatch flow silently changes pickup behavior too.
9. **No dedicated rental-start timestamp.** `delivery_date` / `delivery_time` and `equipment.current_status_changed_at` serve this role implicitly. If any future billing or reporting logic assumes a distinct "rental start" field, it doesn't exist yet.

6. Recommended corrections (minimum set, Phase 1 scope only)

These are the minimum changes to make the *communication path* trustworthy — no lifecycle redesign, no service centralization, no business-logic refactor.

1. **Make the mobile "success" state depend on the actual HTTP response.** Replace the fixed 2-second timer in `CheckListUpdateViewController.swift` with a completion handler tied to the real POST response (`success == "1"`). Show a distinct "Saved — syncing" state if offline, and only show "Checklist updated successfully" once the server has confirmed it. This is the single highest-leverage fix — it directly resolves the primary audit question.
2. **Add dedup/idempotency to the customer checklist queue**, mirroring the pattern already used in `SyncDriverChecklist.swift` / `SyncDeliveryPickupInputs.swift` : remove any existing pending entry for the same `order_product_unique_id` + type before appending a new one, and/or send a client-generated idempotency key the backend can use to reject exact repeats.
3. **Surface dropped submissions.** When a queued checklist exhausts `kMaxSyncAttempts` , notify the user/admin (local notification, badge, or a "needs attention" queue view) instead of silently discarding it.
4. **Auto-trigger Rental Ready eligibility on return.** At minimum, flag the equipment record (e.g., a `needs_rental_ready_inspection` marker) when `markReturnedDamaged` / `markReturnedToMaintenance` runs, so it surfaces on an operational dashboard instead of relying on someone remembering. Full Rental Ready workflow audit is out of scope for Phase 1, but the trigger gap itself is a communication-path gap and belongs here.
5. **Give the pickup path an explicit identity.** Even without changing behavior, name the branch (e.g., a computed `$isPickup` derived from "no prior dispatch," logged/labeled explicitly in `SaveDeliveryController`) so the distinction is visible in code and audit history rather than purely inferred.
6. **Consolidate the redundant audit-history write on delivery/pickup**, or at minimum annotate the two rows so they're distinguishable as one logical event, not two.

Items not recommended for Phase 1: touching `BillingEngine / WaitListMatcher` swallowed-exception behavior (working as intended — return must not be blocked by billing/wait-list failures) or restructuring the Rental Ready service itself (explicitly deferred to the full lifecycle audit).